

Tri-Cloud Vault: Technical Architecture Specification

Intent-Driven Multi-Cloud Storage Optimization & Compensating Rollback Engine

1. Executive Overview & Core Architectural Guarantee

Tri-Cloud Vault is an intent-driven multi-cloud storage optimization platform that transparently places, replicates, and manages data across **Amazon Web Services (AWS S3)**, **Microsoft Azure (Blob Storage)**, and **Google Cloud Platform (GCP Cloud Storage)**.

The Fundamental Design Guarantee:

- Strict Semantic Separation:** The Large Language Model (LLM) acts strictly as a natural language compiler that extracts and validates structured constraints (ParsedConstraints) from user intent. The LLM **never** makes cloud placement decisions directly.
- Deterministic Optimization:** Multi-cloud tier placement is determined exclusively by a mathematical Mixed-Integer Linear Programming (MILP) solver that rigorously minimizes cost or latency subject to SLA constraints.
- Asynchronous Orchestration:** Multi-cloud direct presigned chunk uploads and atomic rollbacks are governed by a 17-state Celery/Redis Finite State Machine (FSM).



2. Repository Architecture & File Manifest

Module Path	Component	Responsibility
backend/.../clouds/	Cloud SDK Adapters	AWS S3, Azure Blob, and GCP Cloud Storage clients, presigned URL generators.
backend/.../intent/	Intent Compiler	Pydantic schemas, Anthropic/OpenAI/Regex fallback parsers, Django ORM audit models.
backend/.../optimizer/	MILP Solver	PuLP optimization formulation, cost matrices, heuristic baseline benchmarks.
backend/.../orchestration/	17-State FSM	Celery tasks, Redis state tracking, parallel upload allocator, compensating rollback.
backend/.../telemetry/	Pricing Matrix	Dynamic storage, operation, and egress pricing telemetry across AWS, Azure, and GCP.
backend/.../experiments/	Research Simulation	Zipfian workload generator, Monte Carlo runner, Pareto sorter, LaTeX/PNG exporter.
frontend/	Web Dashboard	Intent wizard UI, FSM progress bar, cloud badge indicators, cost trade-off visualizer.
infra/	IaC & Automation	Terraform multi-cloud provisioning, Ansible playbooks, Celery systemd units.

3. MILP Mathematical Optimization Formulation

Sets & Indices: Clouds $\mathcal{C} = \{\text{AWS}, \text{AZURE}, \text{GCP}\}$, Storage Tiers $\mathcal{T} = \{\text{standard}, \text{infrequent}, \text{archive}\}$.

Decision Variables: $x_{c,t} \in \{0, 1\} \quad \forall c \in \mathcal{C}, \forall t \in \mathcal{T}$ where $x_{c,t} = 1$ indicates placement in cloud c tier t .

Total Monthly Cost Formulation:

$$\text{Cost}_{c,t} = (S \cdot C^{\text{storage}}_{c,t}) + (W \cdot C^{\text{write}}_{c,t}) + (R \cdot C^{\text{read}}_{c,t}) + (R \cdot S \cdot C^{\text{egress}}_{c,t})$$
where S is file size (GB), W is write operations, R is read operations, and C are unit pricing rates.

Optimization Objectives:

- Cost Minimization:** $\min \sum_c \sum_t \text{Cost}_{c,t} \cdot x_{c,t}$
- Latency Minimization:** $\min \sum_c \sum_t L_{c,t} \cdot x_{c,t}$
- Balanced Objective:** $\min \left[\alpha \frac{\sum_c \sum_t \text{Cost}_{c,t} \cdot x_{c,t}}{\sum_c \sum_t \text{Cost}_{c,t}} + (1-\alpha) \frac{\sum_c \sum_t L_{c,t} \cdot x_{c,t}}{\sum_c \sum_t L_{c,t}} \right]$ with $\alpha = 0.5$

Mathematical Constraints:

- Replica Redundancy:** $\sum_{c \in \mathcal{C}} \sum_{t \in \mathcal{T}} x_{c,t} \geq K \quad (K \in \{1, 2, 3\})$
- Provider Exclusivity:** $\sum_{t \in \mathcal{T}} x_{c,t} \leq 1 \quad \forall c \in \mathcal{C}$ (at most one tier per cloud provider)
- Budget Bound:** $\sum_c \sum_t \text{Cost}_{c,t} \cdot x_{c,t} \leq B$ (if monthly budget B is specified)
- Latency SLA Bound:** $\frac{\sum_c \sum_t L_{c,t} \cdot x_{c,t}}{\sum_c \sum_t x_{c,t}} \leq L_{\text{req}}$ (if max latency L_{req} is specified)
- Geo-Fencing:** $x_{c,t} = 0 \quad \forall c \notin \mathcal{C}_{\text{allowed}}$

4. Dynamic Pricing & Latency Reference Table (India / Asia South)

Provider (Region)	Tier	Storage (\$/GB-mo)	Write (\$/10k)	Read (\$/10k)	Egress (\$/GB)	Latency
AWS (ap-south-1)	Standard	\$0.0230	\$0.050	\$0.0040	\$0.090	45 ms
AWS (ap-south-1)	Infrequent	\$0.0125	\$0.100	\$0.0100	\$0.090	65 ms
AWS (ap-south-1)	Archive	\$0.0040	\$0.050	\$0.0500	\$0.090	180s
Azure (centralindia)	Standard	\$0.0180	\$0.045	\$0.0035	\$0.085	40 ms
Azure (centralindia)	Infrequent	\$0.0100	\$0.090	\$0.0090	\$0.085	60 ms
Azure (centralindia)	Archive	\$0.0020	\$0.100	\$0.0500	\$0.085	300s
GCP (asia-south1)	Standard	\$0.0200	\$0.050	\$0.0040	\$0.080	42 ms
GCP (asia-south1)	Infrequent	\$0.0100	\$0.100	\$0.0100	\$0.080	58 ms
GCP (asia-south1)	Archive	\$0.0025	\$0.050	\$0.0500	\$0.080	240s

5. 17-State Asynchronous FSM & Compensating Rollback

Upload lifecycle is orchestrated asynchronously via Celery and Redis. The state machine guarantees atomic multi-cloud consistency: if any chunk upload or integrity validation fails, a compensating rollback cleans up all allocated cloud resources.

1. INITIALIZED	-> 7. ALLOCATING_PRESIGNED	-> 13. INTEGRITY_CONFIRMED
2. PARSING_INTENT	-> 8. PRESIGNED_READY	-> 14. COMPLETED (Terminal Success)
3. INTENT_PARSED	-> 9. UPLOADING_PARTS	-> 15. ROLLING_BACK
4. OPTIMIZING_POLICY	-> 10. PARTS_UPLOADED	-> 16. ROLLED_BACK (Compensated)
5. POLICY_DETERMINED	-> 11. COMPLETING_SESSIONS	-> 17. FAILED (Fatal Error)
6. PREPARING_CLOUDS	-> 12. VERIFYING_INTEGRITY	

Compensating Rollback Execution:

- AWS S3:** Aborts active multipart upload IDs (s3.abort_multipart_upload) or deletes uploaded objects.
- Azure Blob:** Deletes uncommitted block lists and staging blobs (blob_client.delete_blob).
- GCP Storage:** Deletes incomplete composite chunks and uploaded objects (blob.delete).

6. REST API Endpoint Reference

Endpoint	Method	Description & Payload
/api/intent/parse/	POST	Extracts constraints from natural text. {"user_text": "...", "file_size_bytes": N}
/api/optimizer/solve/	POST	Solves MILP placement model. {"constraints": {...}, "file_size_bytes": N}
/api/orchestration/upload/start/	POST	Initiates async FSM pipeline. Returns operation_id.
/api/orchestration/status/<op_id>/	GET	Polls FSM progress, current state, and direct presigned upload URLs.
/api/orchestration/complete/<op_id>/	POST	Signals chunk upload completion to trigger integrity check & final commit.
/api/telemetry/pricing/	GET	Returns live multi-cloud pricing tables and latency benchmarks.

7. Testing, Benchmarking & Deployment Operations

1. Run Full Unit Test Suite:
python -m unittest discover -s . -p "tests.py" (Verifies intent, optimizer, and FSM modules with 100% pass rate).
2. Execute Research Simulation Suite:
python manage.py run_benchmark --num-workloads 100 --out-dir ./benchmark_results
Generates benchmark_data.csv, publication LaTeX table table_comparison.tex, and Matplotlib figures.
3. Automated Production Cloud Deployment:
 - Provision infrastructure via Terraform: terraform apply -var-file="environments/dev/terraform.tfvars"
 - Deploy application & services via Ansible: ansible-playbook -i inventory/aws_ec2.yml playbook.yml
 - Target Git URL: <https://github.com/charan-sai-ramiseti/Intent-based-cloud-storage.git>
4. Critical Architectural Invariants for Future AI Agents:
 - Never allow an LLM to select cloud providers directly; all requirements must compile into ParsedConstraints.
 - Preserve multi-cloud rollback parity for any newly introduced cloud provider or storage tier.
 - Always use UTC timezone-aware datetimes (datetime.now(timezone.utc)).